

Project Report

On

Heart Disease Prediction Using Python

Submitted in Partial Fulfillment of the Requirements for the Summer Internship at NIELIT Dehradun



Guide by:

Mr. Aditya Karanwal

Submitted by:

Radhika Chaurasiya

Date of Submission: 18\07\2026

Abstract

This project, "**Heart Disease Prediction Using Python**," was developed as part of my Summer Internship in Machine Learning using Python. The objective of the project is to predict the possibility of heart disease in patients by analyzing various medical parameters such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other health-related attributes.

The project is implemented using the Python programming language and Machine Learning techniques. Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn are used for data preprocessing, visualization, model development, and performance evaluation. The dataset is cleaned, analyzed, and divided into training and testing sets before applying Machine Learning algorithms to build a prediction model.

The developed model is evaluated using standard performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix to measure its effectiveness. The results demonstrate that Machine Learning can assist in the early prediction of heart disease with good accuracy, helping healthcare professionals make faster and more informed decisions.

This internship project provided practical experience in data preprocessing, exploratory data analysis, Machine Learning model building, and performance evaluation using Python. It also enhanced my understanding of how Artificial Intelligence and Machine Learning can be applied in the healthcare sector to support early disease detection and improve patient care.

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me during the successful completion of my Summer Internship project titled "**Heart Disease Prediction Using Python.**"

First of all, I would like to thank **NIELIT Dehradun** for providing me with the opportunity to learn **Machine Learning using Python** and gain practical knowledge through this project. The internship helped me understand the concepts of data analysis, machine learning algorithms, and their real-world applications in the healthcare domain.

I am deeply thankful to my project mentor, **Mr. Aditya Karanwal**, for valuable guidance, continuous support, and encouragement throughout the internship. Their suggestions and technical knowledge helped me complete this project successfully.

I would also like to express my heartfelt gratitude to the faculty members of **Government Polytechnic Dehradun** for their constant motivation and support during the preparation of this project report.

Finally, I am grateful to my parents, family, and friends for their encouragement, patience, and moral support throughout the internship and project work.

I sincerely thank everyone who directly or indirectly contributed to the successful completion of this project.

Table of Contents

S. No.	Contents	Page no
1	Introduction	1
2	Objectives of the Project	2
3	Problem Statement	3
4	Scope of the Project	4
5	Literature Review	5
6	Tools and Technologies Used	6-7
7	System Requirements	8
8	Methodology	9-10
9	User Interface Design	11-13
10	Backend Implementation	14-17
11	Frontend Implementation	18-21
12	Machine Learning Model	22
13	Result Analysis and Coding	23-27
14	Limitations	28
15	Conclusion	29
16	References	30

Introduction

Heart disease is one of the leading causes of death across the world and has become a major public health concern. Early detection and timely diagnosis of heart disease can significantly improve the chances of successful treatment and reduce the risk of serious complications. However, identifying heart disease through traditional methods may require extensive medical tests and expert analysis. Therefore, there is a growing need for intelligent systems that can assist healthcare professionals in predicting heart disease quickly and accurately.

Machine Learning is a branch of Artificial Intelligence (AI) that enables computers to learn from data and make predictions without being explicitly programmed. In the healthcare sector, Machine Learning has become an important tool for analyzing medical data and supporting clinical decision-making. By using historical patient data, Machine Learning models can identify hidden patterns and predict the likelihood of diseases with high accuracy.

This project, "**Heart Disease Prediction Using Python**," was developed as part of a **Summer Internship on Machine Learning Using Python**. The main objective of the project is to build a prediction system that can determine whether a person is at risk of heart disease based on various medical attributes such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other related health factors.

The project is implemented using the Python programming language and various libraries, including Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn. The dataset is first collected and preprocessed by handling missing values and preparing the data for analysis. After preprocessing, the data is used to train a Machine Learning model, which is then tested and evaluated using standard performance metrics to measure its prediction accuracy.

The proposed system demonstrates how Machine Learning can support the healthcare industry by providing fast and reliable predictions. Although it cannot replace professional medical diagnosis, it can serve as a valuable decision-support tool for doctors and healthcare providers. This project also provides practical experience in data preprocessing, exploratory data analysis, model building, model evaluation, and the application of Machine Learning techniques using Python.

Overall, the project highlights the importance of Artificial Intelligence and Machine Learning in solving real-world healthcare problems. It also demonstrates how data-driven technologies can improve the quality of medical services, assist in early disease detection, and contribute to better patient care.

Objective of Project

The primary objective of this project is to develop a Machine Learning-based system using Python that can predict the likelihood of heart disease based on a patient's medical information. The project aims to demonstrate how Machine Learning techniques can be applied in the healthcare sector to support early disease detection and improve decision-making.

The specific objectives of this project are as follows:

- To develop a heart disease prediction system using Python and Machine Learning techniques.
- To collect and analyze a heart disease dataset containing various medical attributes such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other health-related parameters.
- To preprocess the dataset by handling missing values, removing unnecessary data, and preparing the data for effective model training.
- To perform exploratory data analysis (EDA) to understand the relationships between different medical attributes and heart disease.
- To train and test a Machine Learning model using the processed dataset for accurate prediction.
- To evaluate the performance of the prediction model using standard evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
- To provide a simple, reliable, and efficient system that can assist healthcare professionals in identifying patients who may be at risk of heart disease.
- To gain practical knowledge of Machine Learning concepts, data preprocessing, model development, data visualization, and performance evaluation using Python.
- To understand the real-world applications of Artificial Intelligence and Machine Learning in the healthcare industry.
- To enhance technical skills in Python programming and build a practical project that demonstrates the use of Machine Learning for solving real-life healthcare problems.

The overall objective of this project is to create an efficient and user-friendly heart disease prediction system that supports early diagnosis and promotes preventive healthcare. The project also aims to provide hands-on experience in implementing Machine Learning techniques using Python as part of the Summer Internship on Machine Learning Using Python.

Problem Statement

Heart disease is one of the most common and life-threatening diseases worldwide. Every year, millions of people are affected by heart-related problems, and many cases result in serious health complications due to late diagnosis. Traditional methods of detecting heart disease often require multiple medical tests, experienced doctors, and considerable time, making the diagnosis process complex and expensive.

With the increasing amount of healthcare data, there is a need for an intelligent system that can analyze patient information and predict the possibility of heart disease at an early stage. Such a system can assist healthcare professionals by providing quick and reliable predictions based on medical parameters such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other clinical attributes.

The main problem addressed in this project is the need for an accurate, efficient, and easy-to-use heart disease prediction system. By applying Machine Learning techniques using Python, the system can analyze historical medical data, identify hidden patterns, and predict whether a patient is likely to have heart disease. This helps reduce the time required for diagnosis, supports clinical decision-making, and promotes early treatment.

Therefore, this project aims to develop a Machine Learning-based heart disease prediction model that can improve prediction accuracy and assist healthcare professionals in making informed decisions. Although the system is not intended to replace medical experts, it can serve as a valuable decision-support tool for the early detection and prevention of heart disease.

Scope of the Project

The scope of this project is to develop a Machine Learning-based system using Python for the early prediction of heart disease. The system analyzes various medical parameters such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other health-related attributes to predict the likelihood of heart disease. The project demonstrates how Machine Learning techniques can be effectively used to support the healthcare sector by providing fast and accurate predictions.

This project mainly focuses on data preprocessing, exploratory data analysis, model development, training, testing, and performance evaluation. It provides practical experience in applying Machine Learning algorithms to real-world healthcare data using Python and its libraries, including Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

The developed prediction system can be used as a decision-support tool for healthcare professionals. It helps in identifying individuals who may be at risk of heart disease, allowing timely medical consultation and preventive care. Although the system is not intended to replace medical diagnosis, it can assist doctors in making quicker and more informed decisions.

The project also provides valuable learning opportunities in data analysis, feature selection, model evaluation, and Machine Learning implementation. It serves as a foundation for developing more advanced healthcare applications that integrate Artificial Intelligence, real-time patient monitoring, and larger medical datasets in the future.

Overall, the scope of this project extends beyond academic learning by demonstrating the practical application of Machine Learning in healthcare. It highlights the potential of intelligent systems to improve disease prediction, reduce diagnosis time, support preventive healthcare, and contribute to better patient outcomes.

Literature Review

Heart disease is one of the leading causes of death worldwide, making early prediction and diagnosis an important area of research. Over the past few years, researchers have increasingly used Machine Learning techniques to predict heart disease by analyzing patient medical data. These techniques help identify patterns and relationships within healthcare data, allowing for faster and more accurate predictions than many traditional methods.

Several studies have shown that Machine Learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Naïve Bayes can be effectively used for heart disease prediction. These algorithms are trained using historical patient records containing medical attributes such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other clinical parameters. After training, the models can predict whether a patient is likely to have heart disease.

Data preprocessing plays a significant role in improving the performance of Machine Learning models. Before training the model, the dataset is cleaned by handling missing values, removing duplicate records, and converting data into a suitable format. Exploratory Data Analysis (EDA) is also performed to understand the distribution of data and identify relationships among different features. Proper preprocessing helps improve the accuracy and reliability of the prediction model.

Python has become one of the most popular programming languages for Machine Learning because of its simplicity and powerful libraries. Libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn provide efficient tools for data preprocessing, visualization, model training, and evaluation. These libraries make it easier to build predictive models and analyze healthcare datasets.

Based on the review of previous studies and existing techniques, it is evident that Machine Learning can significantly improve the early prediction of heart disease. This project builds upon these concepts by developing a heart disease prediction system using Python and Machine Learning. The system aims to provide accurate predictions and serve as a decision-support tool for healthcare professionals while also demonstrating the practical application of Machine Learning in the medical field.

Tools and Technologies Used

The **Heart Disease Prediction Using Python** project was developed using various tools and technologies that support data analysis, machine learning, and model development. These tools helped in data preprocessing, visualization, model training, testing, and performance evaluation.

1. Python

Python is the primary programming language used in this project. It is easy to learn, flexible, and widely used for Machine Learning, Data Science, and Artificial Intelligence applications.

2. Jupyter Notebook

Jupyter Notebook is used as the development environment for writing, executing, and testing Python code. It provides an interactive interface that allows code, output, and documentation to be displayed in a single notebook.

3. Pandas

Pandas is a Python library used for data manipulation and analysis. It helps in reading datasets, cleaning data, handling missing values, and organizing information into rows and columns.

4. NumPy

NumPy is used for numerical computations and mathematical operations. It provides fast and efficient support for arrays, matrices, and statistical calculations.

5. Matplotlib

Matplotlib is a data visualization library used to create graphs, charts, and plots. It helps in understanding trends and patterns within the dataset.

6. Seaborn

Seaborn is a statistical visualization library built on top of Matplotlib. It is used to create attractive and informative visualizations such as heatmaps, distribution plots, and correlation graphs.

7. Scikit-learn

Scikit-learn is the Machine Learning library used to build, train, and evaluate the prediction model. It provides algorithms for classification, data splitting, preprocessing, and performance evaluation.

8. Heart Disease Dataset

A heart disease dataset is used to train and test the Machine Learning model. The dataset contains medical information such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other health-related attributes.

9. Machine Learning Algorithm

A Machine Learning classification algorithm is used to analyze the dataset and predict whether a patient is likely to have heart disease. The model is trained using historical medical data and evaluated using standard performance metrics to measure its prediction accuracy.

These tools and technologies work together to develop an efficient, accurate, and reliable heart disease prediction system. They provide practical experience in implementing Machine Learning concepts using Python and demonstrate the application of Artificial Intelligence in the healthcare sector.

System Requirements

The successful development and execution of the **Heart Disease Prediction Using Python** project require both hardware and software resources. The following system requirements are sufficient for developing, training, and testing the Machine Learning model.

Hardware Requirements

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 4 GB (8 GB Recommended)
- **Storage:** Minimum 100 GB Hard Disk/SSD
- **Display:** 1366 × 768 resolution or higher
- **Keyboard and Mouse:** Standard input devices

Software Requirements

- **Operating System:** Windows 10 or Windows 11
- **Programming Language:** Python 3.x
- **Development Environment:** Jupyter Notebook (Anaconda)
- **Python Libraries:**
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-learn
- **Dataset:** Heart Disease Dataset (CSV Format)

Functional Requirements

- Import and load the heart disease dataset.
- Preprocess the data by handling missing values and preparing it for analysis.
- Train the Machine Learning model using the processed dataset.
- Test the model using unseen data.
- Predict whether a patient is likely to have heart disease.
- Evaluate the model using performance metrics such as accuracy, precision, recall, and F1-score.

Non-Functional Requirements

- The system should provide accurate and reliable predictions.
- The prediction process should be fast and efficient.
- The system should be easy to understand and use.
- The application should be capable of handling healthcare data securely.
- The model should produce consistent results for the given input data.

Methodology

The methodology of this project describes the step-by-step process followed to develop the **Heart Disease Prediction Using Python** system. The project uses Machine Learning techniques to analyze patient health data and predict the possibility of heart disease. The complete methodology consists of data collection, data preprocessing, exploratory data analysis, model development, training, testing, and performance evaluation.

1. Data Collection

The first step of the project is collecting the heart disease dataset. The dataset contains important medical attributes such as age, gender, chest pain type, resting blood pressure, cholesterol level, fasting blood sugar, maximum heart rate, exercise-induced angina, and other clinical features that are useful for predicting heart disease.

2. Data Preprocessing

The collected dataset is cleaned and prepared before applying Machine Learning algorithms. Missing values, duplicate records, and unnecessary data are handled to improve the quality of the dataset. The data is then converted into a suitable format for model training.

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the dataset and identify patterns among different features. Graphs and charts are created using Python libraries such as Matplotlib and Seaborn to visualize the data and study the relationship between medical attributes and heart disease.

4. Feature Selection

Important features that contribute to heart disease prediction are selected from the dataset. Feature selection helps improve the performance of the Machine Learning model by reducing unnecessary information and focusing on the most relevant medical parameters.

5. Model Development

The processed dataset is divided into training and testing datasets. A Machine Learning classification algorithm is then used to train the prediction model. The model learns from historical patient data and identifies patterns that help predict the presence of heart disease.

6. Model Evaluation

After training, the model is tested using the testing dataset. The performance of the model is evaluated using standard evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix. These metrics help determine how effectively the model predicts heart disease.

7. Prediction

Finally, the trained model predicts whether a patient is likely to have heart disease based on the input medical parameters. The prediction system provides quick and reliable results, which can assist healthcare professionals in making informed decisions.

Overall, this methodology provides a systematic approach to developing a Machine Learning-based heart disease prediction system using Python. It demonstrates the complete workflow from data collection to final prediction and highlights the practical application of Machine Learning in the healthcare sector.

User Interface Design

The User Interface (UI) is one of the most important parts of the Heart Disease Prediction System because it allows users to interact with the application easily. A good user interface makes the system simple, attractive, and easy to understand. In this project, the user interface is developed using the **Streamlit** framework in Python. Streamlit is a powerful library that helps in creating web applications without using HTML, CSS, or JavaScript.

The main objective of designing the user interface is to allow users to enter patient health details and receive prediction results in a simple and user-friendly way. The interface is designed in such a manner that even users with little technical knowledge can use the application without any difficulty.

Objectives of User Interface Design

The main objectives of the user interface are:

- To provide a simple and easy-to-use application.
- To collect patient information accurately.
- To reduce user confusion while entering data.
- To display prediction results clearly.
- To make the application responsive and interactive.
- To improve the overall user experience.

Tools Used for User Interface Design

The following tools and technologies are used for designing the interface:

- **Python Programming Language**
- **Streamlit Library**
- **Visual Studio Code / Jupyter Notebook**
- **Pickle Library** (for loading the trained model)

Streamlit provides different functions that help in creating interactive web pages without writing complex frontend code.

Steps Followed to Design the User Interface

Step 1: Creating the Application

The first step was to create a new Python file named **app.py**. This file contains all the code required for the user interface.

Step 2: Importing Required Libraries

The required libraries such as **Streamlit**, **NumPy**, and **Pickle** were imported into the project. These libraries help in creating the interface and loading the trained machine learning model.

Example:

- Streamlit is used for designing the interface.
- Pickle is used to load the saved machine learning model.
- NumPy is used to process user input before prediction.

Step 3: Displaying the Project Title

A title was added at the top of the application using Streamlit.

Example title:

Heart Disease Prediction System Using Machine Learning

This title helps users understand the purpose of the application immediately after opening it.

Step 4: Creating Input Fields

Input fields were created to collect patient information. Every health parameter is entered by the user through text boxes or number input boxes.

The following information is collected:

- Age
- Gender
- Chest Pain Type
- Resting Blood Pressure
- Cholesterol Level
- Fasting Blood Sugar
- Resting ECG
- Maximum Heart Rate
- Exercise-Induced Angina
- Old Peak
- Slope
- Number of Major Vessels
- Thalassemia

Each field accepts only one value from the user.

Step 5: Predict Button

After entering all the required information, the user clicks on the **Predict** button.

The button sends all entered values to the trained machine learning model.

The prediction process starts only after clicking this button.

Step 6: Displaying Prediction Result

Once the prediction is completed, the result is displayed immediately on the screen.

The application displays one of the following messages:

- **The person is likely to have heart disease.**

OR

- **The person is not likely to have heart disease.**

The result is displayed clearly so that users can understand it easily.

Features of the User Interface

The designed user interface contains several useful features.

Simple Design:-The interface has a clean layout without unnecessary buttons or menus.

Easy Navigation:-Users can easily move from one input field to another.

Interactive Interface:-The application immediately responds after the Predict button is clicked.

User Friendly:-No programming knowledge is required to use the application.

Fast Response:-Prediction is generated within a few seconds.

Error Reduction:-The interface ensures that users enter all required information before prediction.

Working of the User Interface

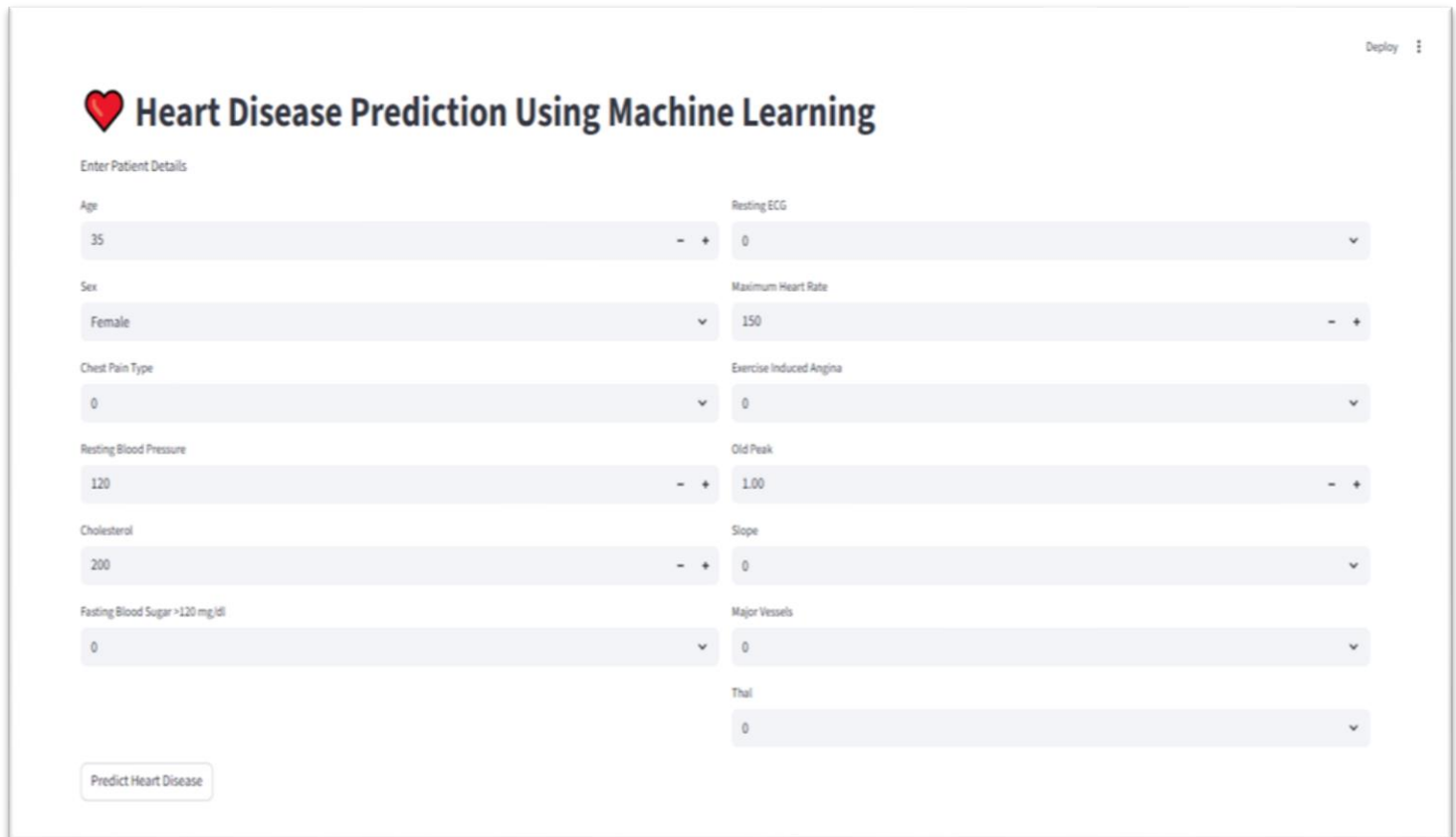
The complete working process of the interface is explained below:

1. The application starts using the Streamlit server.
2. The project title is displayed.
3. The trained machine learning model is loaded automatically.
4. Input fields are shown on the screen.
5. The user enters patient information.
6. The Predict button is clicked.
7. User input is converted into machine-readable format.
8. The trained model processes the input.
9. The prediction result is generated.
10. The final prediction is displayed on the screen.

Advantages of the User Interface Design

- Easy to understand.
- Attractive and simple layout.
- Quick prediction.

- Interactive design.
- No web development knowledge required.
- Works directly in a web browser.
- Saves user time.
- Provides accurate interaction with the machine learning model.
- Easy to modify and update.
- Suitable for beginners as well as professionals.



The screenshot shows a web application titled "Heart Disease Prediction Using Machine Learning" with a heart icon. It features a "Deploy" button in the top right corner. The main section is titled "Enter Patient Details" and contains a grid of input fields for patient information. The fields are arranged in two columns:

- Left Column:** Age (35), Sex (Female), Chest Pain Type (0), Resting Blood Pressure (120), Cholesterol (200), and Fasting Blood Sugar >120 mg/dl (0).
- Right Column:** Resting ECG (0), Maximum Heart Rate (150), Exercise Induced Angina (0), Old Peak (1.00), Slope (0), Major Vessels (0), and Thal (0).

Each input field has a numeric value and a small minus/plus icon for adjustment. At the bottom left, there is a button labeled "Predict Heart Disease".

Backend Implementation

The backend is the core part of the Heart Disease Prediction System. It is responsible for processing user input, loading the trained machine learning model, making predictions, and returning the final result to the user interface. In this project, the backend is developed using the **Python programming language** because Python provides powerful libraries for data processing, machine learning, and web application development.

The backend performs all the logical operations of the application. It receives patient information entered by the user, converts it into the required format, passes it to the trained machine learning model, and finally displays the prediction result through the Streamlit interface.

Objectives of Backend Implementation

The main objectives of the backend are:

- To receive user input from the frontend.
- To process and validate the entered data.
- To load the trained machine learning model.
- To predict whether the patient has heart disease or not.
- To send the prediction result back to the user interface.
- To ensure fast and accurate prediction.

Technologies Used

The backend of the project is implemented using the following technologies:

- **Python Programming Language**
- **Pandas Library**
- **NumPy Library**
- **Scikit-learn Library**
- **Pickle Library**
- **Streamlit Framework**

Each technology performs a specific task in the backend implementation.

Backend Development Process

Step 1: Loading the Dataset

The first step in backend development is loading the heart disease dataset. The dataset contains medical information of patients that is required for training the machine learning model.

The dataset is loaded using the **Pandas** library.

Example:

```
data = pd.read_csv("heart.csv")
```

This command reads the dataset and stores it in a DataFrame for further processing.

Step 2: Separating Features and Target

After loading the dataset, the data is divided into two parts.

Features (X)

These include all the patient health parameters such as:

- Age
- Gender
- Chest Pain Type
- Blood Pressure
- Cholesterol
- ECG

- Maximum Heart Rate
- Thal
- Other medical parameters

Target (Y)

The target column indicates whether the patient has heart disease or not.

Step 3: Training the Machine Learning Model

- The Logistic Regression algorithm is used to train the model.
- The model learns patterns from the dataset and understands the relationship between patient health parameters and heart disease.
- After training, the model becomes capable of predicting disease for new patient data.

Step 4: Saving the Trained Model

- Once the model is trained successfully, it is saved using the **Pickle** library.
- The trained model is stored as: heart_model.pkl
- Saving the model avoids training it every time the application starts.

Step 5: Loading the Saved Model

- Whenever the Streamlit application starts, the backend loads the saved machine learning model.
- This process is completed automatically when the application begins.
- Loading the saved model makes prediction faster and reduces execution time.

Step 6: Receiving User Input

The backend receives patient information entered through the Streamlit interface.

The entered values include:

- Age
- Sex
- Chest Pain Type
- Resting Blood Pressure
- Cholesterol
- Fasting Blood Sugar
- Resting ECG
- Maximum Heart Rate
- Exercise-Induced Angina
- Old Peak
- Slope
- Number of Major Vessels
- Thalassemia

These values are collected and prepared for prediction.

Step 7: Data Processing

- The entered values are converted into the format required by the trained machine learning model.
- The backend ensures that the data is arranged in the correct order before prediction.
- This preprocessing step helps the model produce accurate results.

Step 8: Prediction

- After preprocessing, the patient data is passed to the Logistic Regression model.
- The model analyzes the input data and predicts whether the patient is likely to have heart disease.
- The prediction is generated within a few seconds.

Step 9: Returning the Result

- The prediction result is sent back to the Streamlit interface.
- The application displays one of the following messages:
 1. **High Risk: Heart Disease Detected**
 2. **Low Risk: No Heart Disease Detected**
- The result is displayed instantly for the user.

Backend Workflow

The complete backend workflow is as follows:

1. Load the heart disease dataset.
2. Train the Logistic Regression model.
3. Save the trained model using Pickle.
4. Load the saved model in the application.
5. Receive patient details from the user.
6. Process and validate the input data.
7. Pass the data to the trained model.
8. Generate the prediction.
9. Display the prediction result on the screen.

Features of Backend Implementation

- Fast prediction process.
- Automatic model loading.
- Accurate data processing.
- Efficient handling of user input.
- Secure storage of the trained model.
- Easy integration with the Streamlit interface.
- Reusable machine learning model.
- Simple and maintainable code structure.
- Quick response time.
- Easy to update with a new trained model.

Advantages of Backend Implementation

- Provides fast prediction results.
- Reduces manual calculation.
- Improves prediction accuracy.
- Easy to maintain and update.
- Efficient processing of patient data.
- Supports real-time prediction.
- Can be integrated with web applications.
- Saves execution time by loading the trained model instead of training repeatedly.

Frontend Implementation

The frontend is the visible part of the Heart Disease Prediction System with which users directly interact. It provides a simple and user-friendly interface where users can enter patient health details and receive prediction results. In this project, the frontend is developed using the **Streamlit** framework, which allows developers to create interactive web applications using only Python programming.

The main purpose of the frontend is to collect patient information in an organized manner and send it to the backend for processing. After receiving the prediction from the machine learning model, the frontend displays the result in a clear and understandable format. Since Streamlit automatically creates a web interface from Python code, there is no need to use HTML, CSS, or JavaScript for this project.

Objectives of Frontend Implementation

The main objectives of the frontend are:

- To provide a simple and interactive user interface.
- To collect patient information accurately.
- To make the application easy to use.
- To display prediction results clearly.
- To improve user experience through an attractive layout.
- To connect user input with the machine learning model.

Technologies Used

The frontend is developed using the following technologies:

- **Python Programming Language**
- **Streamlit Framework**
- **Pandas Library**
- **Pickle Library**

These technologies work together to create a responsive and interactive application.

Frontend Development Process

Step 1: Creating the Streamlit Application

The frontend development starts by creating a Python file named **app.py**. This file contains all the code required to build the user interface.

The Streamlit framework is imported into the project, and the application is initialized.

Step 2: Designing the Application Layout

- A clean and attractive layout is created using Streamlit. A project title is displayed at the top of the application, followed by instructions for the user.
- The layout is designed to make navigation simple and easy. Input fields are arranged properly so that users can enter information without confusion.

Step 3: Creating Input Components

The frontend provides different input fields for entering patient details.

The following medical parameters are included:

- Age
- Gender
- Chest Pain Type
- Resting Blood Pressure
- Cholesterol Level
- Fasting Blood Sugar
- Resting ECG
- Maximum Heart Rate
- Exercise-Induced Angina
- Old Peak
- Slope
- Number of Major Vessels
- Thalassemia

Streamlit components such as **Number Input** and **Select Box** are used to collect these values.

Step 4: Predict Button

- A **Predict Heart Disease** button is placed below the input fields.
- When the user clicks this button, all entered information is collected and sent to the backend machine learning model for prediction.
- This button acts as a connection between the frontend and backend.

Step 5: Displaying Prediction Result

- After the backend processes the entered data, the frontend displays the prediction result instantly.
- If the model predicts heart disease, a warning message is displayed.
- If the model predicts no heart disease, a success message is displayed.

- This allows users to understand the prediction quickly.

Step 6: Displaying Dataset Information

- The frontend also displays a preview of the dataset using a table.
- The first few rows of the dataset are shown so that users can understand the type of data used for training the machine learning model.
- A bar chart is also displayed to show the distribution of the target values.
- This improves the presentation of the project.

Working of the Frontend

The frontend follows the following working process:

1. The Streamlit application starts.
2. The project title is displayed.
3. Patient input fields appear on the screen.
4. The user enters all required medical information.
5. The user clicks the **Predict Heart Disease** button.
6. The entered data is sent to the backend.
7. The backend processes the data and returns the prediction.
8. The frontend displays the final prediction result.
9. The dataset preview and chart are also displayed for better understanding.

Features of the Frontend

The frontend provides many useful features, such as:

- Simple and clean interface.
- Easy navigation for users.
- Interactive input fields.
- Fast response after prediction.
- Attractive layout using Streamlit.
- Automatic display of prediction results.
- Dataset preview in tabular format.
- Graphical representation using charts.
- Responsive web interface.
- Easy integration with the backend model.

```

import streamlit as st
import pandas as pd
import pickle
# Page Settings
st.set_page_config(
    page_title="Heart Disease Prediction",
    page_icon="❤️",
    layout="wide")
st.title("❤️ Heart Disease Prediction Using Machine Learning")
# Load Model
model = pickle.load(open("heart_model.pkl", "rb"))
st.write("Enter Patient Details")
col1, col2 = st.columns(2)
with col1:
    age = st.number_input("Age", 20, 100, 35)
    sex = st.selectbox("Sex", ["Female", "Male"])
    cp = st.selectbox("Chest Pain Type", [0,1,2,3])
    trestbps = st.number_input("Resting Blood Pressure", 80, 250, 120)
    chol = st.number_input("Cholesterol", 100, 600, 200)
    fbs = st.selectbox("Fasting Blood Sugar >120 mg/dl", [0,1])
with col2:
    restecg = st.selectbox("Resting ECG", [0,1,2])
    thalach = st.number_input("Maximum Heart Rate", 60, 220, 150)
    exang = st.selectbox("Exercise Induced Angina", [0,1])
    oldpeak = st.number_input("Old Peak", 0.0, 6.0, 1.0)
    slope = st.selectbox("Slope", [0,1,2])
    ca = st.selectbox("Major Vessels", [0,1,2,3,4])
    thal = st.selectbox("Thal", [0,1,2,3])
sex = 1 if sex == "Male" else 0
if st.button("Predict Heart Disease"):
    input_data = [[age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal]]
    prediction = model.predict(input_data)
    if prediction[0] == 1:
        st.error("⚠️ High Risk: Heart Disease Detected")
    else:
        st.success("✅ Low Risk: No Heart Disease Detected")
    st.subheader("Entered Patient Details")
    df = pd.DataFrame(input_data, columns=["Age", "Sex", "Chest Pain", "BP", "Cholesterol", "FBS", "ECG", "Max Heart Rate", "Exercise Angina", "Old Peak", "Slope", "Major Vessels", "Thal"])
    st.dataframe(df)

```

```

# -----
# Dataset Preview
# -----
st.markdown("----")
st.subheader("📊 Dataset Preview")
try:
    data = pd.read_csv("heart2.csv")
    st.dataframe(data.head())
    st.write("Dataset Shape:", data.shape)
    st.subheader("Target Distribution")
    st.bar_chart(data["target"].value_counts())
except Exception as e:
    st.warning(f"Dataset not found: {e}")
# -----
# Footer
# -----
st.markdown("----")
st.markdown(
    "<center><h4>❤️ Heart Disease Prediction Using Machine Learning</h4>"
    "<p>Developed by Radhika Chaurasiya</p></center>",
    unsafe_allow_html=True)

```

Machine Learning Model

The Machine Learning model is the core component of the **Heart Disease Prediction Using Python** project. It is responsible for analyzing the medical data of patients and predicting whether a person is likely to have heart disease. The model is developed using Python and trained on a heart disease dataset containing various medical attributes such as age, gender, blood pressure, cholesterol level, chest pain type, fasting blood sugar, maximum heart rate, and other clinical features.

After the dataset is preprocessed, it is divided into two parts: the training dataset and the testing dataset. The training dataset is used to teach the Machine Learning model by allowing it to learn patterns and relationships from historical patient records. Once the training process is completed, the testing dataset is used to evaluate the model's performance on new and unseen data.

The Machine Learning model identifies patterns in the input data and classifies whether a patient is at risk of heart disease. During the training process, the model continuously learns from the available data to improve its prediction capability. The performance of the model is measured using evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix to ensure reliable and consistent predictions.

The trained model provides fast and efficient predictions based on the medical information provided by the user. Although the model cannot replace the expertise of medical professionals, it serves as a valuable decision-support tool that can assist doctors in the early detection of heart disease.

Overall, the Machine Learning model demonstrates the practical application of Artificial Intelligence and Machine Learning in the healthcare sector. It highlights how data-driven techniques can improve disease prediction, support timely diagnosis, and contribute to better patient care.

Result analysis

From these results we can see that although most of the researchers are using different algorithms such as SVC, Decision tree for the detection of patients diagnosed with Heart disease, KNN, Random Forest Classifier and Logistic regression yield a better result to out rule them.

The algorithms that we used are more accurate, saves a lot of money i.e. it is cost efficient and faster than the algorithms that the previous researchers used. Moreover, the maximum accuracy obtained by KNN and Logistic Regression are equal to 88.5% which is greater or almost equal to accuracies obtained from previous researches.

So, we summarize that our accuracy is improved due to the increased medical attributes that we used from the dataset we took. Our project also tells us that Logistic Regression and KNN outperforms Random Forest Classifier in the prediction of the patient diagnosed with a heart Disease.

CODING

After an extensive introduction, we can finally perform heart disease detection in Python using a hands-on tutorial that implements several machine learning algorithms, primary exploratory data analysis, and inbuilt data analysis techniques for feature importance. To test out how different models perform on the task, we will use the UCI Heart Disease dataset having 14 columns and over 300 samples. You can find the dataset on.

Heart Disease Prediction UCI Dataset and download the CSV file.

1. Import jumpy as np
2. Import pandas as pd
3. Import matplotlib.pyplot as plt
4. From sklearn.model_selection import train_test_split, RandomizedSearchCV
5. From sklearn.preprocessing import StandardScaler
6. From sklearn.neighbors import KNeighborsClassifier
7. From sklearn.svm import SVC
8. From sklearn.tree import DecisionTreeClassifier
9. From sklearn.ensemble import
10. Random Forest Classifier, Gradient Boosting Classifier

First, we must load Python's necessary libraries crucial for data science. We use Sci-kit Learn to implement the k neighbours' classifier, SVM, Random Forest, and Decision Tree.

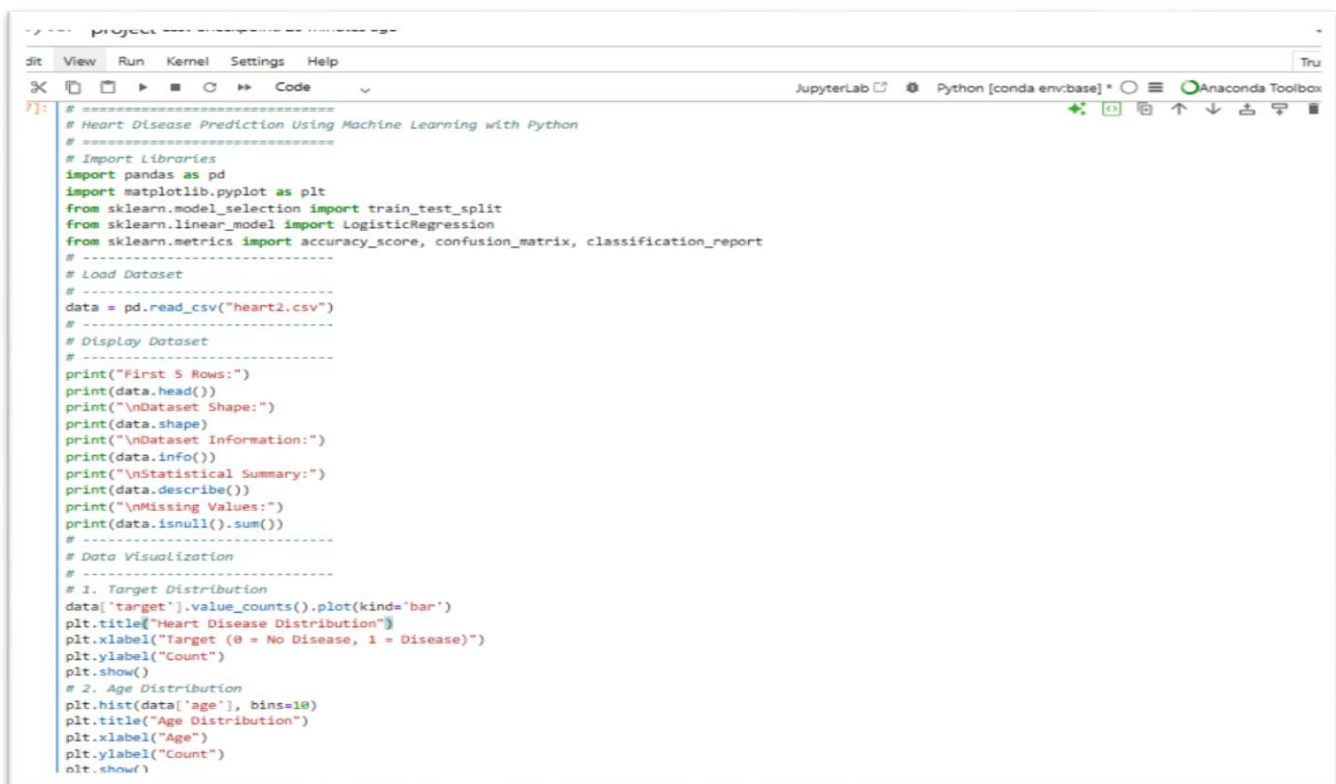
Implementations from Sci-kit learn are well optimized with the option for tuning. It also lets you perform a Grid Search to find the best set of hyper parameters for a model.

The search is performed using module in Sci-kit learn. Prediction of Heart Disease using Machine Learning Algorithms.

Next, we load the dataset CSV file using Pandas. The Pandas package efficiently loads and access large datasets directly from an external file and performs data processing, feature engineering, and data analysis. In our case, we will use the read_csv function to load our file.

Heart Disease Prediction using Machine Learning Python.

INPUT:-



```

# Heart Disease Prediction Using Machine Learning with Python
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Dataset
data = pd.read_csv("heart2.csv")

# Display Dataset
print("First 5 Rows:")
print(data.head())
print("\nDataset Shape:")
print(data.shape)
print("\nDataset Information:")
print(data.info())
print("\nStatistical Summary:")
print(data.describe())
print("\nMissing Values:")
print(data.isnull().sum())

# Data Visualization
# 1. Target Distribution
data['target'].value_counts().plot(kind='bar')
plt.title("Heart Disease Distribution")
plt.xlabel("Target (0 = No Disease, 1 = Disease)")
plt.ylabel("Count")
plt.show()

# 2. Age Distribution
plt.hist(data['age'], bins=10)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()

```

```
plt.show()
# 3. Cholesterol Distribution
plt.hist(data['chol'], bins=10)
plt.title("Cholesterol Distribution")
plt.xlabel("Cholesterol")
plt.ylabel("Count")
plt.show()
# 4. Gender Distribution
data['sex'].value_counts().plot(kind='bar')
plt.title("Gender Distribution")
plt.xlabel("Sex (0 = Female, 1 = Male)")
plt.ylabel("Count")
plt.show()
# 5. Chest Pain Type
data['cp'].value_counts().plot(kind='bar')
plt.title("Chest Pain Type")
plt.xlabel("Chest Pain Type")
plt.ylabel("Count")
plt.show()
# -----
# Features and Target
# -----
X = data.drop("target", axis=1)
Y = data["target"]
# -----
# Split Dataset
# -----
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
# -----
# Model Training
# -----
model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)
# -----
# Prediction
# -----
Y_pred = model.predict(X_test)
```

```
# -----
Y_pred = model.predict(X_test)
# -----
# Accuracy
# -----
accuracy = accuracy_score(Y_test, Y_pred)
print("\nModel Accuracy = ", round(accuracy*100,2), "%")
# Accuracy Graph
plt.bar(["Accuracy"], [accuracy*100])
plt.title("Model Accuracy")
plt.ylabel("Accuracy (%)")
plt.ylim(0,100)
plt.show()
# -----
# Confusion Matrix
# -----
print("\nConfusion Matrix:")
print(confusion_matrix(Y_test, Y_pred))
# -----
# Classification Report
# -----
print("\nClassification Report:")
print(classification_report(Y_test, Y_pred))
# -----
# Predict New Patient
# -----
input_data = (52,1,0,125,212,0,1,168,0,1,0,2,2,3)
input_df = pd.DataFrame([input_data], columns=X.columns)
prediction = model.predict(input_df)
print("\nPrediction Result:")
if prediction[0] == 1:
    print("The person is likely to have Heart Disease.")
else:
    print("The person is not likely to have Heart Disease.")
```

OUTPUT:-

```

JupyterLab Python [conda env:base] Anaconda Toolbox

First 5 Rows:
  age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52   1   0      125   212    0         1     168    0       1.0    2
1   53   1   0      140   203    1         0     155    1       3.1    0
2   70   1   0      145   174    0         1     125    1       2.6    0
3   61   1   0      148   203    0         1     161    0       0.0    2
4   62   0   0      138   294    1         1     106    0       1.9    1

  ca  thal  target
0   2    3      0
1   0    3      0
2   0    3      0
3   1    3      0
4   3    2      0

Dataset Shape:
(1025, 14)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   age         1025 non-null  int64
1   sex         1025 non-null  int64
2   cp          1025 non-null  int64
3   trestbps    1025 non-null  int64
4   chol        1025 non-null  int64
5   fbs         1025 non-null  int64
6   restecg     1025 non-null  int64
7   thalach     1025 non-null  int64
8   exang       1025 non-null  int64
9   oldpeak     1025 non-null  float64
10  slope       1025 non-null  int64
11  ca          1025 non-null  int64
12  thal        1025 non-null  int64
13  target      1025 non-null  int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
None

```

```

JupyterLab Python [conda env:base] Anaconda Toolbox

Statistical Summary:

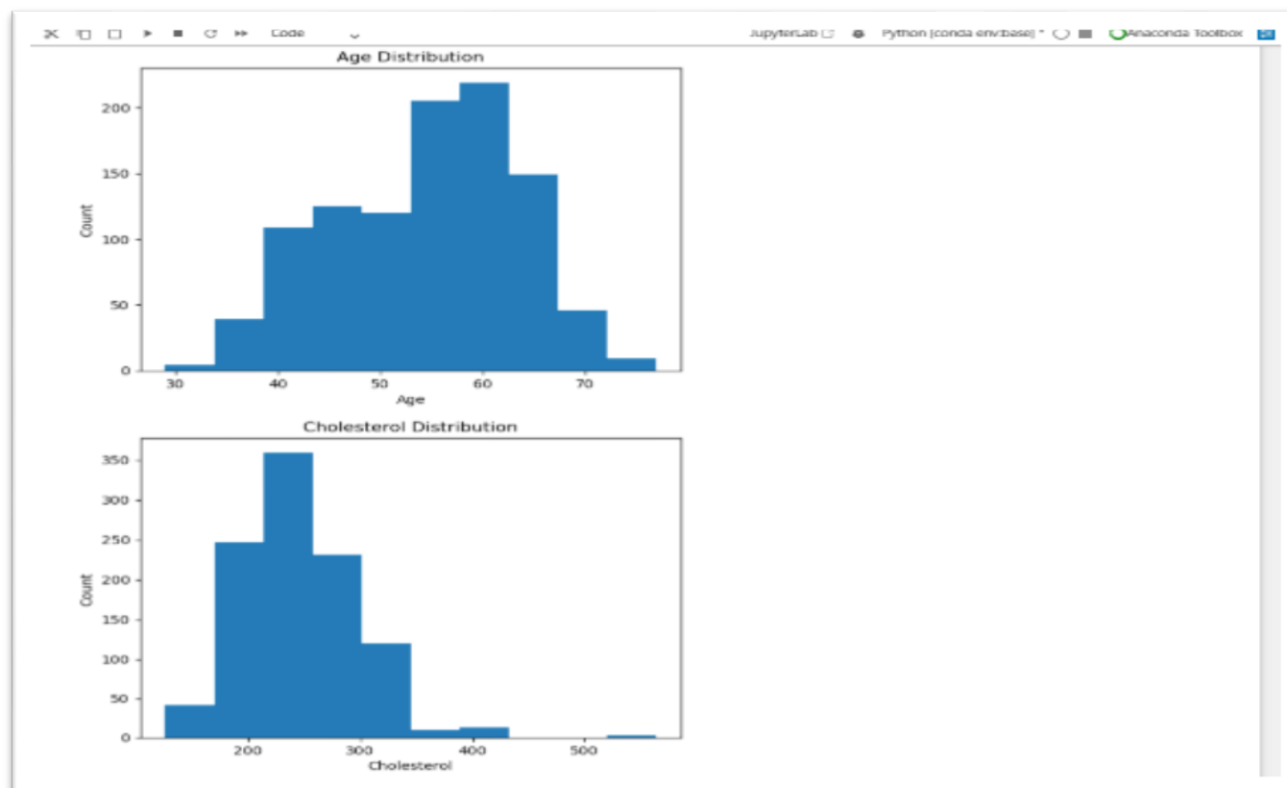
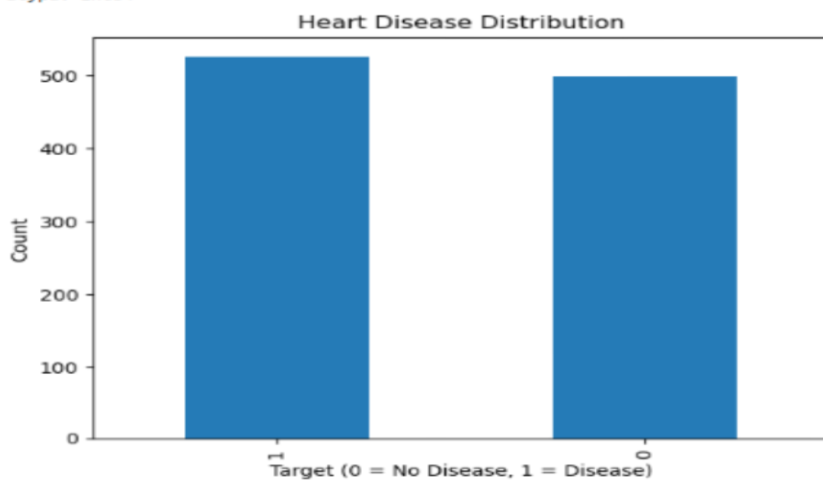
  age      sex      cp      trestbps      chol  \
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000
mean    54.434146    0.695610    0.942439   131.611707   246.000000
std     9.072290    0.460373    1.029641   17.516718    51.59251
min     29.000000    0.000000    0.000000    94.000000   126.000000
25%     48.000000    0.000000    0.000000   120.000000   211.000000
50%     56.000000    1.000000    1.000000   130.000000   240.000000
75%     61.000000    1.000000    2.000000   140.000000   275.000000
max     77.000000    1.000000    3.000000   200.000000   564.000000

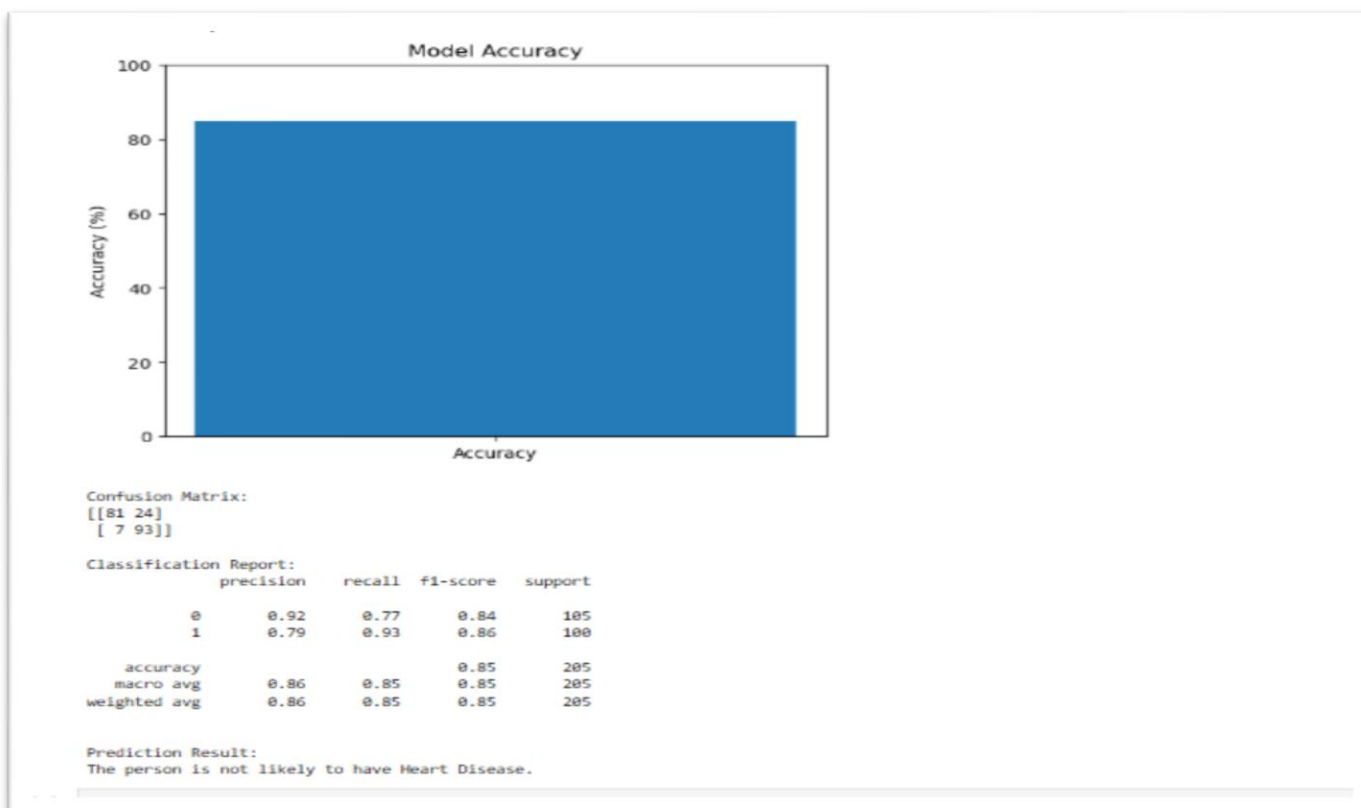
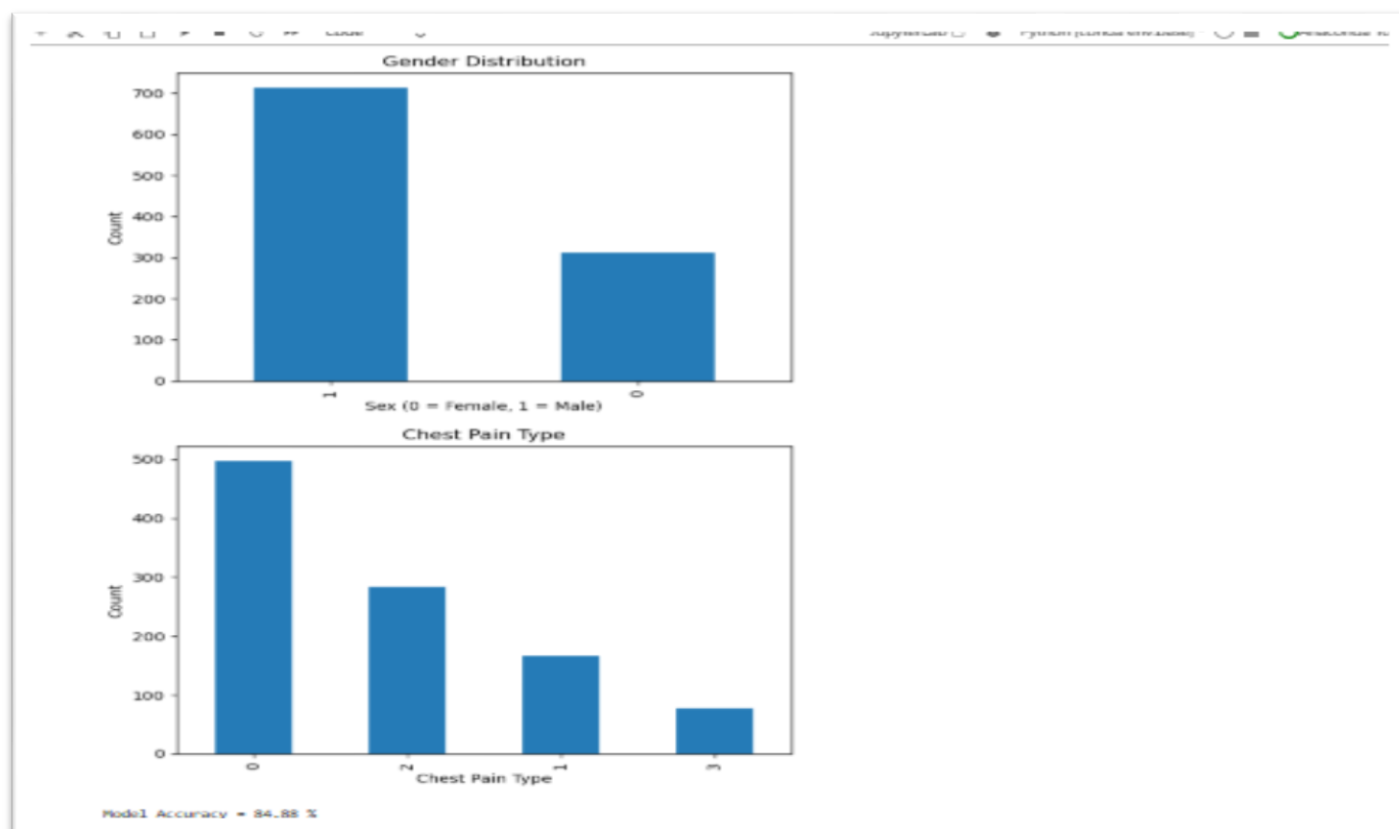
  fbs      restecg      thalach      exang      oldpeak  \
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000
mean    0.149268    0.529756   149.114146    0.336585    1.071512
std     0.356527    0.527878   23.005724    0.472772    1.175053
min     0.000000    0.000000    71.000000    0.000000    0.000000
25%     0.000000    0.000000   132.000000    0.000000    0.000000
50%     0.000000    1.000000   152.000000    0.000000    0.800000
75%     0.000000    1.000000   166.000000    1.000000    1.800000
max     1.000000    2.000000   202.000000    1.000000    6.200000

  slope      ca      thal      target
count  1025.000000  1025.000000  1025.000000  1025.000000
mean    1.385366    0.754146    2.323902    0.513171
std     0.617755    1.030798    0.620660    0.500070
min     0.000000    0.000000    0.000000    0.000000
25%     1.000000    0.000000    2.000000    0.000000
50%     1.000000    0.000000    2.000000    1.000000
75%     2.000000    1.000000    3.000000    1.000000
max     2.000000    4.000000    3.000000    1.000000

```

```
Missing Values:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```





Limitations

Although the Heart Disease Prediction Using Python system is useful and effective, it has some limitations that should be considered:

1. **Depends on Data Quality:** The accuracy of the prediction depends on the quality and completeness of the dataset. Incorrect or missing data can lead to inaccurate results.
2. **Limited Dataset:** If the model is trained on a small or outdated dataset, it may not perform well for all patients or different populations.
3. **Not a Replacement for Doctors:** The system provides only a prediction and cannot replace the diagnosis or advice of a qualified medical professional.
4. **Prediction Errors:** Machine learning models may sometimes produce false positive or false negative predictions, which can affect decision-making.
5. **Limited Features:** The prediction is based only on the available input features. Other important medical factors, such as family history, lifestyle, and additional laboratory reports, may not be included.
6. **Requires Regular Updates:** The model should be retrained with new medical data to maintain accuracy and improve performance over time.
7. **No Real-Time Monitoring:** The system predicts heart disease based on the entered data but does not continuously monitor a patient's health condition.
8. **Performance Depends on Algorithm:** Different machine learning algorithms may produce different levels of accuracy. Logistic Regression may not always provide the best performance for every dataset.
9. **Data Privacy Concerns:** Patient medical information must be stored and processed securely to protect privacy and confidentiality.
10. **Technical Knowledge Required:** Basic knowledge of Python, machine learning, and data handling is required to develop, maintain, or modify the system.

In conclusion, the Heart Disease Prediction Using Python project is a valuable tool for predicting the risk of heart disease, but it should be used only as a decision-support system. Final diagnosis and treatment should always be made by qualified healthcare professionals.

Conclusion

The **Heart Disease Prediction Using Python** project demonstrates how machine learning can be used to predict the likelihood of heart disease based on a patient's medical information. By using Python and the Logistic Regression algorithm, the system analyzes important health parameters such as age, blood pressure, cholesterol level, heart rate, and other clinical features to make predictions.

The model is trained using historical medical data and tested to evaluate its performance. The prediction results are generated quickly and with good accuracy, making the system useful for identifying patients who may be at risk of heart disease. This helps support early diagnosis, timely medical consultation, and preventive healthcare measures.

The project also highlights the practical application of machine learning in the healthcare sector. It reduces manual effort, provides faster analysis, and assists doctors in making informed decisions. Although the system has certain limitations and should not replace professional medical diagnosis, it can serve as an effective decision-support tool.

In conclusion, the Heart Disease Prediction Using Python project is an efficient, reliable, and cost-effective solution for predicting heart disease risk. With further improvements, larger datasets, and advanced machine learning techniques, the system can become more accurate and beneficial for both healthcare professionals and patients.

References

1. UCI Machine Learning Repository. Heart Disease Dataset. <https://archive.ics.uci.edu/ml/datasets/heart+disease>
2. Kaggle. Heart Disease Dataset. <https://www.kaggle.com/datasets>
3. Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
4. Pandas Documentation. <https://pandas.pydata.org/docs/>
5. Scikit-learn Documentation. <https://scikit-learn.org/stable/>
6. NumPy Documentation. <https://numpy.org/doc/>
7. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
8. Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press.
9. World Health Organization (WHO). Cardiovascular Diseases. <https://www.who.int>
10. American Heart Association. Heart Disease Information. <https://www.heart.org>